

TokenOptimizer

Architectural Design Document

Token-optimisation platform for AI-assisted development — layered, offline-first,
and measurable.

Version 0.1.0 • 15 Jul 2026 • Status: Baseline

Document Control

Field	Value
Title	TokenOptimizer — Architectural Design Document
Version	0.1.0
Date	15 Jul 2026
Status	Baseline
Audience	Engineers, architects, and reviewers
Scope	Full system: optimizer engine, feature-dev skills, interfaces, evaluation
Related	docs/FEATURES.md, docs/CHANGELOG.md, README.md

Contents

- 1. Introduction & Purpose
- 2. Architectural Goals & Constraints
- 3. System Context
- 4. Logical Architecture (Layered View)
- 5. Component Responsibilities
- 6. Feature-Development Skills
- 7. Key Runtime Flows
- 8. Cross-Cutting Concerns
- 9. Evaluation & Validation Architecture
- 10. Deployment & Runtime View
- 11. Technology Stack
- 12. Key Design Decisions (Rationale)
- 13. Quality Attributes
- 14. Directory Structure
- 15. Risks & Future Work

1. Introduction & Purpose

TokenOptimizer reduces the number of tokens sent to a Large Language Model, cutting cost and latency per call while preserving every fact a downstream agent needs. Beyond a general text optimizer, it ships two **feature-development skills** — PRD compression and codebase anchoring + model routing — with an offline A/B suite that proves the cost/quality outcome.

This document describes the system's architecture: its layers, components, runtime flows, cross-cutting concerns, and the design decisions behind them. The guiding principle is **offline-first**: the deterministic strategies and both skills run with no API key; a Claude key or a local model only improves quality, and is never required.

2. Architectural Goals & Constraints

Goals

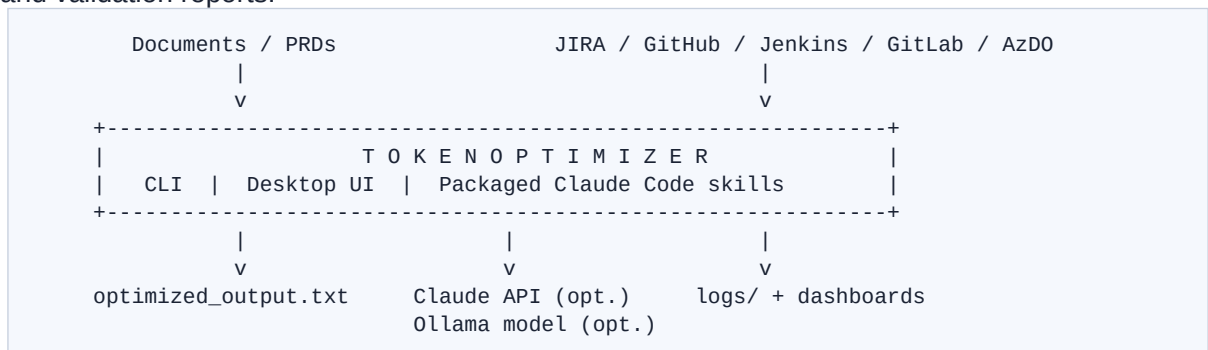
- Reduce LLM token spend measurably (input compression, routing, caching).
- Preserve decision-critical content — never silently drop facts or acceptance criteria.
- Work fully offline; degrade gracefully when optional dependencies are absent.
- Ground AI output in verifiable code (file:line) to eliminate hallucination rework.
- Be observable and reproducible: per-run logs and deterministic evaluation.

Constraints

- Python 3.10+; minimal hard dependencies (anthropic, httpx, python-dotenv, python-docx).
- Desktop UI must be dependency-free (standard-library Tkinter only).
- No secrets in logs or output; credentials only via environment / .env.
- Optional extras: tiktoken (exact offline counts), reportlab (docs), Ollama (local model).

3. System Context

TokenOptimizer sits between a user (or automation) and external systems. Inputs are documents, JIRA issues, GitHub PRs, or PRDs; outputs are optimized text, anchored plans, routing decisions, run logs, and validation reports.



4. Logical Architecture (Layered View)

The package is organised into layers. Higher layers depend on lower ones; the core layer is depended upon by everything and depends on nothing else in the project.

```

+-----+
| INTERFACES      cli.py      .   ui/  (Tkinter desktop app) |
+-----+
| SKILLS (top-level plugins)  prd_compression .             |
|                             codebase_anchoring . model_routing |
| WORKFLOWS              automations/ (triage, review)         |
| EVALUATION              evaluation/ (ab_runner, rubric, cost, dashboard)|
+-----+
| OPTIMIZE          local . compress . summarize . prefilter . |
|                  tokens . text/structured pipelines         |
+-----+
| INTEGRATIONS      jira . github . gitlab . jenkins . azdo . docs |
+-----+
| CORE              config . run_log . llm (client + cache) |
+-----+

```

- **Core** — cross-cutting infrastructure: configuration, per-run logging, Claude client + response cache.
- **Integrations** — turn external systems and files into plain text.
- **Optimize** — the reduction strategies and the pipelines that compose them.
- **Skills / Workflows / Evaluation** — feature-dev skills, ready-made automations, and the A/B harness.
- **Interfaces** — CLI, desktop UI, and packaged Claude Code skills.

5. Component Responsibilities

Package	Responsibility
core/config.py	Env-driven configuration (models, keys, cache/log dirs).
core/run_log.py	Write one structured, secret-free record per run.
core/llm/	Claude client (count_tokens, adaptive thinking) + disk response cache.
optimize/	Deterministic reductions, compression, summarization tiers, token counting, and the text/structured pipelines.
integrations/	Connectors for JIRA, GitHub, GitLab, Jenkins, Azure DevOps, and documents.
automations/	End-to-end flows: JIRA triage, Jenkins RCA, GitHub PR review/triage.
skills/prd_compression/	Skill 1 — compress a PRD into requirement atoms (top-level plugin).
skills/codebase_anchoring/	Skill 2a — index a repo and anchor plan steps to file:line.
skills/model_routing/	Skill 2b — classify task complexity and route to a model.
evaluation/	A/B runner, 25-point rubric, cost model, dataset, HTML dashboard.
ui/	Tkinter desktop app (three tabs) — app.py, python -m entry.
cli.py	The tokenopt command exposing every capability.

6. Feature-Development Skills

Skill 1 — PRD Compression (skills/prd_compression)

Segments a PRD into units, classifies each into a requirement category (goal, acceptance-criteria, constraint, non-functional, dependency, out-of-scope), drops low-signal framing, and re-renders terse

atoms. Acceptance criteria are preserved verbatim. ~67–73% input-token reduction.

Skill 2a — Codebase Anchoring (skills/codebase_anchoring)

Indexes a repository into a symbol table with exact file:line locations (AST for Python, regex for JS/TS/Java/Go/Ruby/C#) and resolves each plan step's symbol mentions to a real anchor. Unresolved symbol-like terms are flagged as possible hallucinations.

Skill 2b — Model Routing (skills/model_routing)

Classifies a task as trivial / standard / complex with a confidence score, then routes to haiku / sonnet / opus respectively. A confidence-threshold fallback upgrades one tier toward premium when the classifier is unsure, so a cheap model is never chosen on a coin-flip.

7. Key Runtime Flows

7.1 Document / structured optimize flow

```
raw text -> deterministic reductions -> summarization (optional tier)
        -> whitespace/dedupe compress -> optimized text + token report -> log
```

7.2 Feature-development flow (the two skills + validation)

```
PRD/spec --> compress_prd --> requirement atoms --> plan_fn(LLM) --> plan steps
                                     |
build_index(repo) -----> anchor_plan -----> + file:line anchors
                                     |
route_task(task) -----> model tier -----> cheapest capable model
                                     |
score_quality + estimate_cost --> A/B metrics --> dashboard (HTML / native)
```

8. Cross-Cutting Concerns

Concern	Approach
Configuration	core/config.py reads env / .env; blanks fall back to defaults.
Logging	core/run_log.py writes a per-run report + JSON; never raises; secrets as booleans.
Caching	core/llm/cache.py disk cache for identical calls; Claude native prompt cache for stable prefixes.
Token counting	optimize/tokens.py: API count_tokens -> tiktoken -> heuristic.
Error handling	Graceful degradation at every optional boundary; nothing hard-fails offline.
Security	No secrets in logs or output; credentials only via env / .env.

9. Evaluation & Validation Architecture

The evaluation package proves the cost/quality claims deterministically and offline. For each of 8 PRDs across 2 business units it runs a baseline arm (raw PRD -> premium model, no anchoring) against an optimised arm (compressed -> anchored -> routed) and scores both on a 25-point rubric.

- ab_runner.py — orchestrates both arms and aggregates deltas.
- quality_rubric.py — correctness, completeness, anchoring, actionability, no-hallucination.

- cost.py — tokens x per-model list price.
- datasets.py — 8 realistic verbose PRDs across Payments and Platform BUs.
- dashboard.py — self-contained HTML (inline SVG) rendered from the results.

Validated, test-locked outcome: ~73% average PRD compression, ~58% average cost savings, 24.0/25 average optimised quality (baseline 19.5).

10. Deployment & Runtime View

- Installed as an editable package; the tokenopt console script is the primary entry point.
- Desktop UI: tokenopt ui or python -m token_optimizer.ui (Tkinter, no extra deps).
- Windows launcher run.bat wraps the venv for double-click or terminal use.
- Each skill is a self-contained top-level package under skills/ (code + SKILL.md + tests).
- Runs on a single machine; no server component. Optional network calls: Claude API, Ollama, integrations.

11. Technology Stack

Area	Choice
Language / runtime	Python 3.10+
Packaging	setuptools src-layout; console_scripts entry point
LLM SDK	anthropic (Claude); Messages API + prompt caching
HTTP	httpx (integration connectors)
Documents	python-docx
Token counting	tiktoken (optional), API count_tokens, heuristic fallback
Desktop UI	Tkinter (standard library)
Docs / PDF	reportlab (optional extra)
Local model	Ollama HTTP server (optional)
Testing	pytest (offline, no API key)

12. Key Design Decisions (Rationale)

Decision	Rationale
Offline-first pipeline	Real token savings without a key; API/local model only improves quality.
core / skills separation	Isolate cross-cutting infrastructure from product skills; clear dependency direction.
Deterministic skills & rubric	Reproducible results and testable claims; no model-in-the-loop nondeterminism.
Confidence-threshold routing	Under-powering a hard task costs more than the routing saved; upgrade when unsure.

Decision	Rationale
Anchoring flags, not silently accepts	Surfacing unresolved references eliminates hallucination-driven rework.
src-layout package	Prevents accidental imports of the working tree; standard for distributable packages.
Self-contained HTML dashboard	Zero runtime chart dependency; portable evidence for any stakeholder.
Skip-if-noop stages	Reported stages reflect what actually changed the text; easier to reason about.

13. Quality Attributes

- **Reliability** — graceful degradation at every optional boundary; logging can't break a run.
- **Security** — secrets never logged or emitted; credentials only via env / .env.
- **Testability** — 58 offline tests; headline claims locked by dedicated tests.
- **Performance / cost** — pre-filtering and compression are 0-token; routing and caching cut spend.
- **Maintainability** — layered packages, single-responsibility modules, relative imports.
- **Observability** — structured per-run logs (human + JSON) and an A/B dashboard.

14. Directory Structure

src/token_optimizer/	core package
cli.py	tokenopt command (all subcommands)
core/	config . run_log . llm (client + cache)
optimize/	local . compress . summarize . prefilter . tokens . text/structured pipelines
integrations/	jira . github . gitlab . jenkins . azdo . docs
automations/	trriage . github review
evaluation/	ab_runner . quality_rubric . cost . datasets . dashboard
ui/	app.py (three-tab Tkinter app), __main__.py
skills/	self-contained skill plugins (code + SKILL.md + tests)
prd_compression/	Skill 1 . codebase_anchoring/ Skill 2a
model_routing/	Skill 2b
docs/	FEATURES.md . CHANGELOG.md . this ADD . plan . PDFs
examples/	sample PRD / document inputs
tests/	test_optimize.py . test_hackcelerate.py

15. Risks & Future Work

- PRD compression is tuned for PRD-shaped input; raw Jira bug tickets suit the structured pipeline better.
- The A/B plan generator is a deterministic stub; a live LLM plan_fn can be plugged in without harness changes.
- Regex symbol indexing (non-Python languages) is heuristic; a tree-sitter backend could improve accuracy.
- Cost figures use list prices; wire real usage metering for production dashboards.
- Consider persisting dashboards and trend history across runs for longitudinal reporting.